전북대학교
JEONBUK NATIONAL UNIVERSITY

리눅스 프로그래밍
(Linux Programming)

2024 - 1 학기

15. Parallel Programming

Jongwook Jeong
정 종 욱

Division of Computer Science and Engineering

Contents

- Parallel Programming

- Server

Parallel Programming

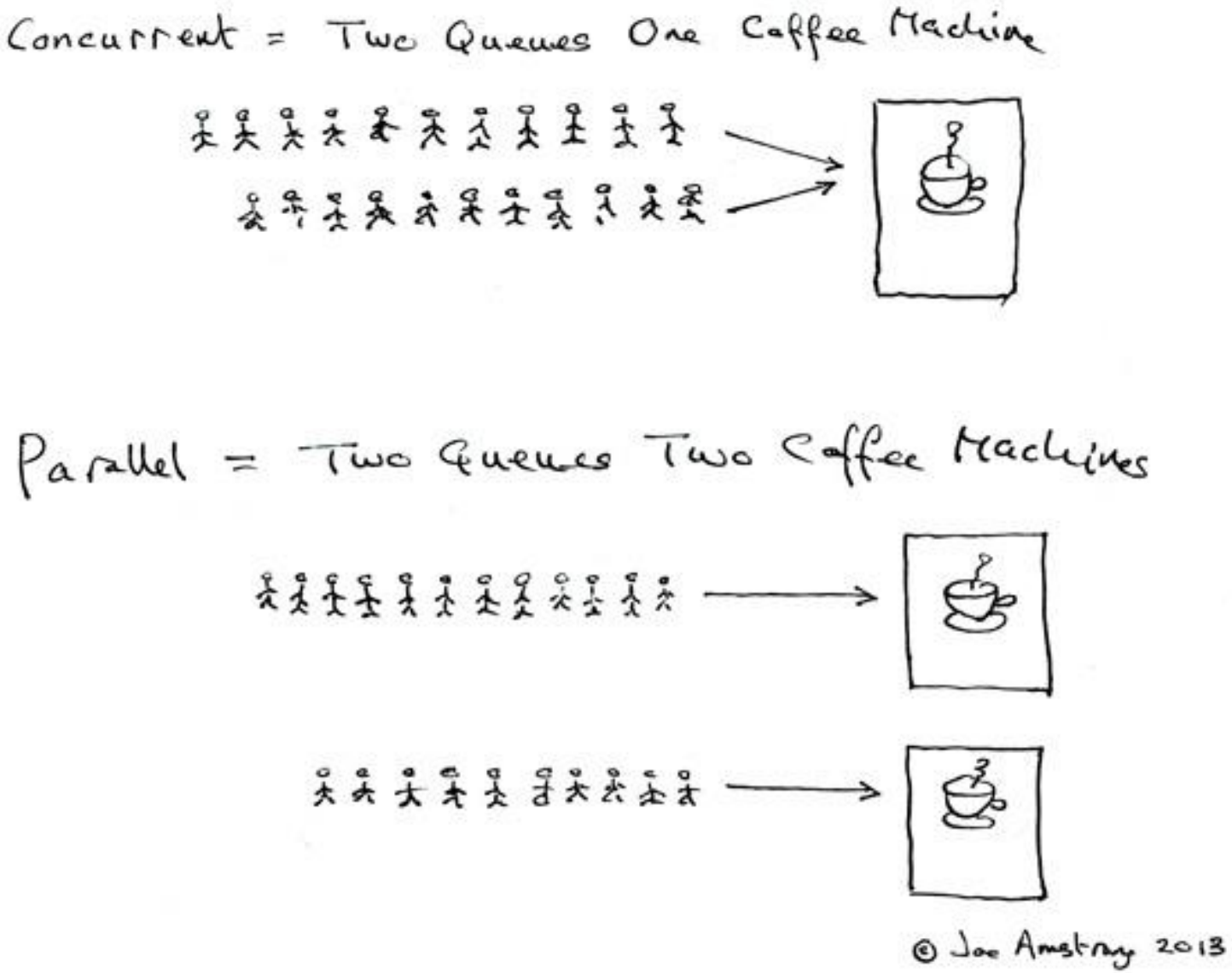
병렬 프로그래밍

- **공동의 목적을 달성하기 위해, 다수의 실행 주체가 동시에 작업을 수행하는 방식**
  - 실행 주체: Process or Thread
  - 사실 컴퓨터 시스템에서 프로그램을 실행하는 주체는 CPU (or Core)!!
  - 프로세스나 쓰레드는 여러 프로그램이 CPU를 공유해서 사용하기 위한 추상적 단위

- **병렬 프로그래밍의 필요성**
  - 최근 컴퓨터 시스템은 대부분 다수의 CPU 를 탑재하고 있으며,
  - 여러 CPU를 최대한 활용하여 성능을 높이기 위해서는 병렬 프로그래밍이 필수

MODEL	CORES/ THREADS
AMD Ryzen™ 9 5950X	16C/32T
AMD Ryzen™ 9 5900X	12C/24T
AMD Ryzen™ 7 5800X	8C/16T
AMD Ryzen™ 5 5600X	6C/12T

3rd Generation Intel Xe				
Model	Family	L3 Cache	Cores	Threads
8380HL	Platinum	38.50	28	56
8380H	Platinum	38.50	28	56
8376HL	Platinum	38.50	28	56
8376H	Platinum	38.50	28	56
8354H	Platinum	24.75	18	36
8353H	Platinum	24.75	18	36
6348H	Gold	33.00	24	48
6328HL	Gold	22.00	16	32
6328H	Gold	22.00	16	32
5320H	Gold	27.50	20	40
5318H	Gold	24.75	18	36



https://joearms.github.io/published/2013-04-05-concurrent-and-parallel-programming.html

소켓 프로그래밍에서는?

- 일반적으로 서버는 여러 클라이언트에 대해 동시에 서비스를 제공함
  - 예) 포털에 여러 사람이 동시에 접근하여 각자 서로 다른 서비스를 이용함

- 반복 서버
  - 하나의 프로세스가 동작하며 모든 클라이언트의 요청을 순서대로 처리
  - 클라이언트는 연결 수립을 위해 이전 클라이언트의 요청이 모두 처리될 때까지 대기하여야 함

- 동시 동작 서버 (Parallel or Concurrently running or Multi-user Server)
  - 여러 요청을 동시에 처리해 서비스 할 수 있는 서버
  - 서버를 병렬로 동작하도록 구성
  - 일반적인 서버 역할 분담
    - Door-keeper process: 문지기 역할. 접속을 대기하다, 요청이 오면 연결을 수립함. 그리고 수립된 각각의 연결에 대해 서비스 프로세스를 할당함. 이를 반복함.
    - Service thread: 각 연결에 대해 1:1로 실제 서비스를 제공하는 쓰레드

반복 서버의 예

- 지난 소켓에서의 서버-클라이언트 예제 동작 방식
  - Server: listen()→ accept() → send() → recv() → close()
  - Client: connect() → recv() → send() → close()

- 서버 프로세스 수정: 반복 처리 서버
  - 하나의 클라이언트 처리가 종료된 후, 다시 accept()로 돌아와서 대기
  - 위 동작을 3번 반복하여, 3개의 클라이언트 요청을 처리하게 함 (3개의 client가 요청한 경우)

- 만약 클라이언트가 send()를 하지 않는다면?
  - 서버 프로세스는 recv() 에서 무한 대기하며,
    해당 클라이언트로부터 패킷이 전송되길 기다릴 것

- 그때 만약 다른 클라이언트가 동시에 접속을 요청한다면?
  - 서버가 accept()를 대기하고 있기 않기 때문에, 접속 처리를 할 수 없음

[예제 1] 반복 서버: Server (1/2)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define BUFFSIZE 4096
#define SERVERPORT 7799

int main(void){
        int i, s_sock, c_sock;
        struct sockaddr_in server_addr, client_addr;
        socklen_t c_addr_size;
        char buf[BUFFSIZE] = {0};
        char hello[] = "Hello, I am Server!\n";

        s_sock = socket(AF_INET, SOCK_STREAM, 0);

        int option = 1;
        setsockopt(s_sock, SOL_SOCKET, SO_REUSEADDR, &option, sizeof(option));

        bzero(&server_addr, sizeof(server_addr));

        server_addr.sin_family = AF_INET;
        server_addr.sin_port = htons(SERVERPORT);
        server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

        if(bind(s_sock, (struct sockaddr *) &server_addr, sizeof(server_addr)) == -1){
                perror("[Server] Failed to bind a socket.");
                exit(1);
        }

        listen(s_sock,1);
        c_addr_size = sizeof(struct sockaddr);
```

```
ubuntu@41983:~/hw12$ ./n1
[S] Can't bind a socket: Address already in use
```

```
ubuntu@41983:~/hw12$ netstat -al | grep 7799
tcp        0      0 41983:7799              41983:38752             TIME_WAIT
tcp        0      0 41983:7799              41983:38756             TIME_WAIT
tcp        0      0 41983:38754             41983:7799              TIME_WAIT
```

- 프로그램 종료 후, 다시 실행시켰을 때, bind() 에서 "already in use" 오류가 나는 것을 무시하기 위해 사용 (참고)

[예제 1] 반복 서버: Server (2/2)

```
        for(i=0; i<3; i++){
                printf("[Server] waiting for a client...#%02d\n", i);
                c_sock = accept(s_sock, (struct sockaddr *)&client_addr, &c_addr_size);

                if (c_sock == -1){
                        perror("[Server] Cant` accept a connetcion");
                        exit(1);
                }
                printf("[Server] Connected: client IP addr =%s port=%d\n", inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));


                //1. say hello to client.
                if (send(c_sock, hello, sizeof(hello)+1, 0) == -1){
                        perror("[Server] Failed to send message");
                        exit(1);
                }

                printf("[Server] I said Hello to Client!\n");

                //2. recv msg from client.
                if(recv(c_sock, buf, BUFFSIZE, 0)==-1){
                        perror("Can`t receive message");
                        exit(1);
                }

                printf("[Server] Client says: %s\n",buf);

                close(c_sock);
        }
        close(s_sock);

        return 0;
}
```

[예제 1] 반복 서버: Client (1/2)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define BUFFSIZE 4096
#define SERVERPORT 7799

int main(void){
        int c_sock;
        struct sockaddr_in server_addr, client_addr;
        socklen_t c_addr_size;
        char buf[BUFFSIZE] = {0};
        char hello[] = "Hi, I am Client~~!\n";

        c_sock = socket(AF_INET, SOCK_STREAM, 0);

        bzero(&server_addr, sizeof(server_addr));

        server_addr.sin_family = AF_INET;
        server_addr.sin_port = htons(SERVERPORT);
        server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

        printf("[Client] Connecting...\n");

        if(connect(c_sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) == -1){
                perror("Failed to connect a server.");
                exit(1);
        }
```

[예제 1] 반복 서버: Client (2/2)

```
        printf("[Client] Connected!\n");

        //1. recv msg from server.
        if (recv(c_sock, buf, BUFFSIZE, 0) == -1){
                perror("Failed to receive message");
                exit(1);
        }

        printf("[Client] Read Socket Data. Server Says: %s\n", buf);

        //2. say hi to server.
        if(send(c_sock, hello, sizeof(hello)+1, 0)==-1){
                perror("[Client] Can`t send message");
                exit(1);
        }

        printf("[Client] I said Hi to Server~~!");


        close(c_sock);

        return 0;
}
```

[예제 1] 정상 실행 예

```
root@jjw:/home/jjw/socket# ./s2 &
[1] 834
root@jjw:/home/jjw/socket# [Server] waiting for a client...#00
```

```
root@jjw:/home/jjw/socket# ./c2 & ./c2 & ./c2
[2] 836
[3] 837
[Client] Connecting...
[Server] Connected: client IP addr =127.0.0.1 port=46764
[Server] I said Hello to Client!
[Client] Connected!
[Client] Read Socket Data. Server Says: Hello, I am Server!

[Server] Client says: Hi, I am Client~~!

[Server] waiting for a client...#01
[Client] I said Hi to Server~~!root@jjw:/home/jjw/socket# [Client] Connecting...
[Client] Connecting...
[Server] Connected: client IP addr =127.0.0.1 port=46766
[Server] I said Hello to Client!
[Client] Connected!
[Client] Read Socket Data. Server Says: Hello, I am Server!

[Server] Client says: Hi, I am Client~~!

[Server] waiting for a client...#02
[Client] I said Hi to Server~~![Server] Connected: client IP addr =127.0.0.1 port=46768
[Server] I said Hello to Client!
[Client] Connected!
[Client] Read Socket Data. Server Says: Hello, I am Server!

[Server] Client says: Hi, I am Client~~!

[Client] I said Hi to Server~~!
```

[예제 1] 비정상 실행 예

- 클라이언트 소스 수정
  - Send() 하지 않고, 10초간 sleep()
  - 맨 첫 클라이언트만 수정한 코드로 실행

- 결과
  - 다른 클라이언트들은 연결 수립은 됨
    - 포트가 열려있으므로, 커널이 수락한 것
  - 그러나 recv() 에서 계속 대기하여야 함
    - 서버 프로세스가 recv()에서 대기하느라, send()를 할 수 없으므로
  - 10초 후, 첫 번째 클라이언트가 소켓을 닫음으로써 연결이 종료되고, 이로 인해 recv() 가 취소되어 다음 동작을 수행 가능함

```
root@jjw:/home/jjw/socket# echo a & echo b & echo c &
[1] 1215
[2] 1216
[3] 1217
root@jjw:/home/jjw/socket# c
b
a
```

```
45      //2. say hi to server
46      /*
47      if (send(c_sock, hello, sizeof(hello)+1, 0) == -1) {
48          perror("[C] Can't send message");
49          exit(1);
50      }
51
52      printf("[C] I said Hi to Server!!\n");
53      */
54
55      printf("[C] I am going to sleep...\n");
56      sleep(10);
57
58      close(c_sock);
```

```
root@jjw:/home/jjw/socket# ./s2 &
[1] 1170
root@jjw:/home/jjw/socket# [Server] waiting for a client...#00
```

```
root@jjw:/home/jjw/socket# ./c2 & ./c2 & ./c_err
[2] 1171
[3] 1172
[Client] Connecting...
[Server] Connected: client IP addr =127.0.0.1 port=46866
[Server] I said Hello to Client!
[Client] Connected!
[Client] Read Socket Data. Server Says: Hello, I am Server!

[Client] I`m going to sleep~~~
[Client] Connecting...
[Client] Connecting...
[Client] Connected!
[Client] Connected!
[Server] Client says:
[Server] waiting for a client...#01
[Server] Connected: client IP addr =127.0.0.1 port=46868
[Server] I said Hello to Client!
root@jjw:/home/jjw/socket# [Client] Read Socket Data. Server Says: Hello, I am Server!

[Server] Client says: Hi, I am Client~~!

[Server] waiting for a client...#02
[Server] Connected: client IP addr =127.0.0.1 port=46870
[Server] I said Hello to Client!
[Client] I said Hi to Server~~![Client] Read Socket Data. Server Says: Hello, I am Server!

[Server] Client says: Hi, I am Client~~!

[Client] I said Hi to Server~~!
[1]   Done                    ./s2
[2]-  Done                    ./c2
[3]+  Done                    ./c2
```

Process-based Parallel Socket Programming

- 프로세스 기반 동시 동작 서버
  - Door-keeper process
    - 기존 서버 코드와 같이 서버 소켓을 열고 bind(), listen() 수행
    - 새로운 연결이 수립되면, child process를 생성
      - Child process -> 실제 service를 지원하는 process (메시지 주고받기)
    - 서버는 fork() 이후, 즉각 다시 accept()에서 대기. 이를 3회 반복함
    - 종료 전, 생성된 child 개수만큼 wait()를 수행하여 모든 종료 상태값을 출력 후, 종료
  - Service process
    - 기존과 같이 send(), recv() 수행하고, 소켓을 닫고, 종료

[예제 2] 동시 동작 서버 수행 예제

```
ubuntu@41983:~/hw12$ ./server
[S] waiting for a client..#00
[S] Connected: client IP addr=10.0.0.249 port=42254
[S] waiting for a client..#01
[SS] Service: I am your child! pid=3052751
[SS] I said Hello to Client!
[S] Connected: client IP addr=10.0.0.249 port=42256
[S] waiting for a client..#02
[SS] Service: I am your child! pid=3052752
[SS] I said Hello to Client!
[SS] Client says: Hi~I am Client!!

[S] Connected: client IP addr=10.0.0.249 port=42258
[S] Child #00 is finished with 0
[SS] Service: I am your child! pid=3052753
[SS] I said Hello to Client!
[SS] Client says: Hi~I am Client!!

[S] Child #01 is finished with 0
[SS] Client says:
[S] Child #02 is finished with 0
ubuntu@41983:~/hw12$
```

```
ubuntu@41983:~/hw12$ ./n2_s & ./n2 & ./n2
[2] 3052748
[3] 3052749
[C] Connecting...
[C] Connected!
[C] Server says: Hello~I am Server!

[C] I am going to sleep...
[C] Connecting...
[C] Connected!
[C] Server says: Hello~I am Server!

[C] I said Hi to Server!!
[1]   Done                    ./n2_s
[C] Connecting...
[C] Connected!
[C] Server says: Hello~I am Server!

[C] I said Hi to Server!!
[3]+  Done                    ./n2
ubuntu@41983:~/hw12$
```